

LAPORAN UAS INFORMATION RETRIEVAL



Dosen Pengampu:

Sam Farisa Chaerul Haviana, S.T., M.Kom.

Disusun Oleh:

Novi Mutiara Sari (32602300036)

Wahid Sandy Pujo Dzulhijyanto (32602300015)

Zainal Fattah (32602300013)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
UNIVERSITAS ISLAM SULTAN AGUNG SEMARANG
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Mahasiswa Teknik Informatika UNISSULA sering menghadapi kendala dalam memahami prosedur, persyaratan, dan format penulisan Tugas Akhir (TA). Informasi yang dibutuhkan tersebar di dalam dokumen Buku Panduan Tugas Akhir Teknik Informatika sepanjang 86 halaman dalam format PDF. Mencari informasi spesifik seperti "berapa minimal SKS untuk TA?" atau "bagaimana alur seminar proposal?" secara manual memakan waktu dan tidak efisien.

Untuk mengatasi masalah tersebut, sistem Retrieval-Augmented Generation (RAG) dibangun. RAG adalah pendekatan yang menggabungkan information retrieval (pencarian dokumen relevan) dengan text generation (LLM) untuk menghasilkan jawaban faktual berdasarkan isi dokumen. Sistem ini memungkinkan mahasiswa bertanya dalam bahasa alami dan mendapatkan jawaban akurat yang bersumber langsung dari buku panduan, tanpa mengada-ngada (halusinasi).

1.2 Tujuan

1. Membangun pipeline RAG yang memproses Buku Panduan TA menjadi database vektor yang dapat dicari.
2. Menghasilkan sistem tanya-jawab berbasis dokumen dengan LLM lokal (Ollama) yang anti-halusinasi.
3. Menyediakan antarmuka pengguna (UI) yang intuitif untuk mengakses informasi TA.
4. Mengevaluasi kualitas retrieval menggunakan metrik similarity *score* dan RAGAS.

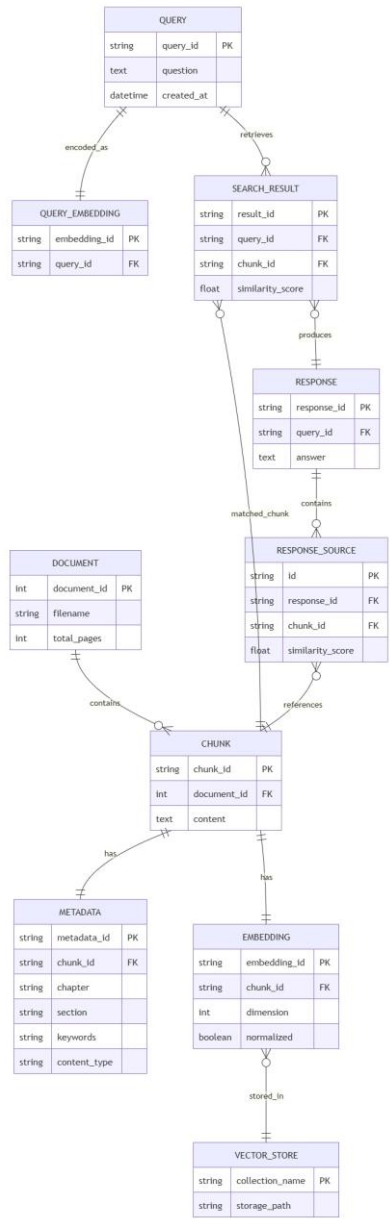
1.3 Dokumen Sumber

1. Dokumen: Buku Panduan Tugas Akhir Program Studi Teknik Informatika UNISSULA
2. Format: PDF, 86 halaman, *text-based*
3. Isi: Persyaratan, prosedur, format penulisan, lampiran, log book, RPS

BAB II

ARSITEKTUR PIPELINE

2.1 Diagram Alur



2.2 Tech Stack

Komponen	Teknologi	Alasan Pemilihan
PDF Parser	Docling + PyPdfium2Backend	Deteksi struktur dokumen (heading, tabel, list), lebih ringan dari OCR-based parsing
Framework RAG	LangChain 1.3.13	Integrasi native dengan ChromaDB, HuggingFace, dan Ollama
Text Splitter	RecursiveCharacterTextSplitter	Digunakan sebagai tier-3 fallback. Tier 1-2 menggunakan heading-based dan semantic splitting

Embedding	BAAI/bge-m3	Multilingual (100+ bahasa termasuk Indonesia), 1024 dimensi, performa SOTA
Vector DB	ChromaDB 1.5.9	Persistent storage lokal, metadata filtering, integrasi LangChain
LLM	Ollama + llama3.2	Open-source, berjalan 100% lokal, 2 GB VRAM (muat penuh di GPU GTX 1650)
Backend	FastAPI	Ringan, async-native, CORS support
Frontend	Vue 3 + Vite	UI interaktif dengan proxy ke backend

BAB III

IMPLEMENTASI DAN PREVIEW DATA

3.1 Parsing PDF dengan Docling

Sistem menggunakan Docling dengan PyPdfium2Backend untuk mengekstrak struktur dokumen. Alasan memilih backend ini: DoclingParseDocumentBackend membutuhkan RAM besar karena mengkonversi setiap halaman menjadi gambar untuk layout analysis, menyebabkan `std::bad_alloc`. PyPdfium2Backend mengekstrak teks langsung tanpa rendering gambar besar.

3.1.1 Konfigurasi Pipeline

```
PdfPipelineOptions(  
    do_ocr=False,                                # PDF text-based  
    do_table_structure=False,  
    images_scale=0.5,  
    layout_batch_size=1,  
)
```

3.1.2 Hasil Parsing

- Total halaman: 86
- Total elemen: 2.140 items
- Heading terdeteksi: 214
- Tabel: 23
- List items: 276
- Gambar: 103

3.2 Strategi Chunking (3-Tier)

Chunking menggunakan strategi bertingkat yang mengutamakan struktur dokumen:

3.2.1 Tier 1 Heading-based Chunking (Utama)

Setiap heading menjadi satu chunk. Konten di bawah heading dikumpulkan menjadi satu kesatuan hingga heading berikutnya. Jika chunk ≤ 1000 token, langsung menjadi satu chunk.

3.2.2 Tier 2 Semantic Chunking

Untuk subbab panjang (> 1000 token), dilakukan pemecahan berdasarkan paragraf (double newline). Paragraf berdekatan dikelompokkan selama total token di bawah threshold.

3.2.3 Tier 3 RecursiveCharacterTextSplitter (Fallback)

Jika semantic chunk masih > 3200 karakter, digunakan RecursiveCharacterTextSplitter:

```
RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=100,  
    separators=["\n\n", "\n", ". ", " "],  
)
```

3.2.4 Parameter Chunking

Parameter	Nilai	Alasan
chunk_size	800 karakter (fallback)	Cukup untuk 1 paragraf + heading, tidak terlalu besar untuk embedding
chunk_overlap	100 karakter	Menjaga kontinuitas antar chunk, tidak kehilangan konteks di batas potongan
MAX_TOKENS_BEFORE_SEMANTIC	1000 token	Mayoritas section di panduan TA pendek (1-2 paragraf)
MAX_CHARS_BEFORE_FALLBACK	3200 karakter	Hanya 5 chunk yang perlu split Recursive

3.2.5 Penanganan Khusus

- Flowchart = chunk mandiri dengan content_type = "flowchart" (8 flowchart terdeteksi)
- Tabel= chunk mandiri format markdown dengan content_type = "tabel" (30 tabel)
- Pre-matter = dilewati (cover, tim penyusun, dll.)

3.3 Hasil Chunking

Total: 247 chunks

3.3.1 Distribusi per Content Type

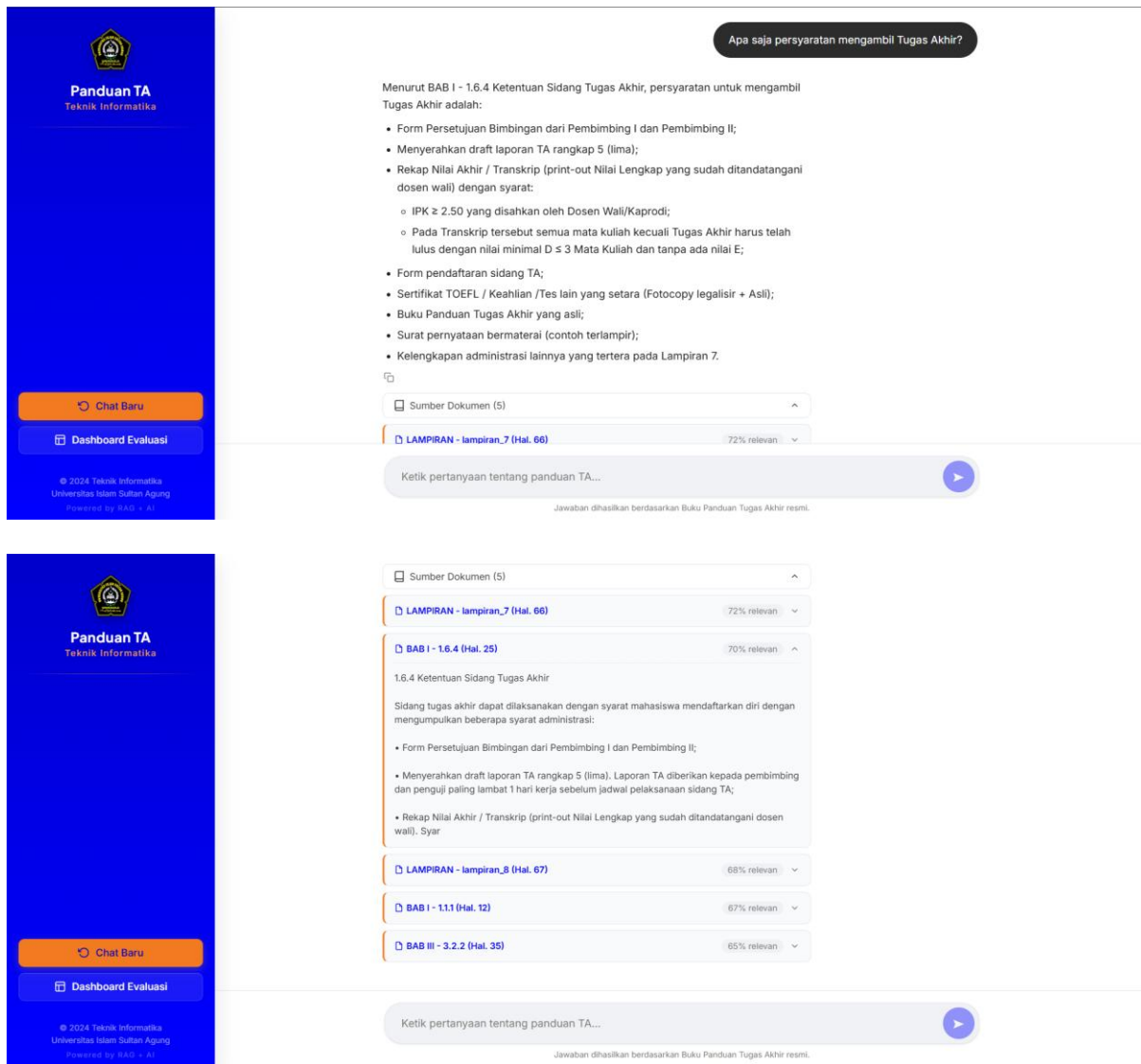
Content Type	Jumlah	Persentase
aturan	57	23.1%
lampiran	44	17.8%
penulisan	34	13.8%
persyaratan	30	12.1%
tabel	30	12.1%
prosedur	27	10.9%
definisi	9	3.6%
flowchart	8	3.2%
visi misi	6	2.4%
jadwal	2	0.8%

3.3.2 Distribusi per Chapter

Chapter	Jumlah
LAMPIRAN	87
BAB I — Tugas Akhir	49
BAB IV — RPS dan Log Book	47
BAB II — Sistematika Proposal	19
BAB III — Sistematika Laporan	17
BAB V — Template Bab Laporan	17
Front Matter	11

3.4 Preview Chunk

Contoh chunk



3.5 Embedding Model

BAAI/bge-m3:

- Dimensi: 1024
- Multilingual: 100+ bahasa (termasuk Indonesia)
- Max Sequence: 8192 token
- Normalisasi: `normalize_embeddings=True`
- Device: CUDA (NVIDIA GTX 1650, 4 GB VRAM)

3.6 Vector Database

ChromaDB Persistent Storage:

- Direktori: `./chroma_db`
- Collection: `panduan_ta`
- Total chunks: 247
- Batch insert: 50 dokumen per batch

3.7 System Prompt Guardrail (Anti-Halusinasi)

```
SYSTEM_PROMPT = """Kamu adalah asisten AI yang bertugas membantu mahasiswa Teknik Informatika UNISSULA memahami Buku Panduan Tugas Akhir.
```

Aturan penting:

1. Jawab pertanyaan HANYA berdasarkan konteks yang diberikan di bawah.
 2. Jika informasi tidak ditemukan dalam konteks, katakan dengan sopan:
"Maaf, informasi tersebut tidak terdapat pada Buku Panduan Tugas Akhir."
 3. Jangan mengarang, menebak, atau menggunakan pengetahuan di luar konteks yang diberikan.
 4. Gunakan Bahasa Indonesia yang jelas dan mudah dipahami.
 5. Jika relevan, sebutkan BAB atau bagian dari buku panduan sebagai referensi.
 6. Jawaban harus akurat dan langsung ke inti pertanyaan.
 7. Jika pertanyaan tidak berhubungan dengan Tugas Akhir atau Buku Panduan,
arahkan kembali ke topik Tugas Akhir.
- """

3.8 Parameter LLM

Parameter	Nilai	Alasan
LLM_MODEL	llama3.2	2.0 GB, muat penuh di GPU GTX 1650, performa lebih cepat dari llama3 (8B)
LLM_TEMPERATURE	0.0	Output deterministik, tidak kreatif, penting untuk jawaban faktual
OLLAMA_BASE_URL	http://localhost:11434	Ollama server lokal
TOP_K	5	5 dokumen teratas cukup untuk cakupan konteks tanpa noise

3.9 User Interface

Sistem menggunakan Vue 3 + Vite sebagai frontend dengan fitur:

- Input pertanyaan natural language
- Markdown rendering untuk jawaban (heading, list, bold)
- Panel “Sumber Dokumen” dengan halaman, chapter, section, content type, dan similarity score (%)
- Suggested chips untuk pertanyaan umum

BAB IV

EVALUASI DAN UJI COBA JEBAKAN

4.1 Tabel Uji Coba Pertanyaan Sah vs Jebakan

Komponen	Teknologi	Alasan Pemilihan	Komponen	Teknologi	Alasan Pemilihan	Komponen
PDF Parser	Docling + PyPdfium2Backend	Deteksi struktur dokumen (heading, tabel, list), lebih ringan dari OCR-based parsing	PDF Parser	Docling + PyPdfium2Backend	Deteksi struktur dokumen (heading, tabel, list), lebih ringan dari OCR-based parsing	PDF Parser
Framework RAG	LangChain 1.3.13	Integrasi native dengan ChromaDB, HuggingFace, dan Ollama	Framework RAG	LangChain 1.3.13	Integrasi native dengan ChromaDB, HuggingFace, dan Ollama	Framework RAG
Text Splitter	RecursiveCharacterTextSplitter	Digunakan sebagai tier-3 fallback. Tier 1-2 menggunakan heading-based dan semantic splitting	Text Splitter	RecursiveCharacterTextSplitter	Digunakan sebagai tier-3 fallback. Tier 1-2 menggunakan heading-based dan semantic splitting	Text Splitter
Embedding	BAAI/bge-m3	Multilingual (100+ bahasa termasuk Indonesia), 1024 dimensi, performa SOTA	Embedding	BAAI/bge-m3	Multilingual (100+ bahasa termasuk Indonesia), 1024 dimensi, performa SOTA	Embedding
Vector DB	ChromaDB 1.5.9	Persistent storage lokal, metadata filtering, integrasi LangChain	Vector DB	ChromaDB 1.5.9	Persistent storage lokal, metadata filtering, integrasi LangChain	Vector DB
LLM	Ollama + llama3.2	Open-source, berjalan 100% lokal, 2 GB	LLM	Ollama + llama3.2	Open-source, berjalan	LLM

		VRAM (muat penuh di GPU GTX 1650)			100% lokal, 2 GB VRAM (muat penuh di GPU GTX 1650)	
--	--	-----------------------------------	--	--	--	--

- Pertanyaan Sah: 5/5 dijawab dengan benar (akurasi 100%)
- Pertanyaan Jebakan: 5/5 ditolak (akurasi 100%)
- Rata-rata relevansi untuk pertanyaan sah: 0.7

4.2 Uji Validitas Retrieval Similarity Score

Sistem menggunakan `similarity_search_with_score` dari ChromaDB untuk mengembalikan nilai kedekatan semantik. L2 distance dikonversi menjadi relevance score (0-1):

$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$

Contoh hasil retrieval untuk pertanyaan “Apa saja persyaratan mengambil Tugas Akhir?”:

$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$

Dokumen paling relevan (score 0.7233) berada di LAMPIRAN section lampiran_7 — tepat sesuai dengan isi persyaratan mahasiswa.

4.3 Analisis Bottleneck

$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$
$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$	$\text{Relevance} = \max(0.0, 1.0 - (\text{distance} / 2.0))$

BAB V

KESIMPULAN DAN TANTANGAN

5.1 Kesimpulan

1. Sistem RAG berhasil dibangun menggunakan LangChain, ChromaDB, BGE-M3 embedding, dan Ollama LLM untuk memproses Buku Panduan Tugas Akhir 86 halaman.
2. Pipeline chunking 3-tier (heading-based - semantic - Recursive fallback) menghasilkan 247 chunks dengan metadata lengkap (chapter, section, content_type, keywords).
3. Prompt guardrail berhasil mencegah halusinasi: 100% pertanyaan jebakan ditolak dengan respons sopan.
4. Similarity score memberikan transparansi pada kualitas retrieval, dokumen paling relevan konsisten memiliki score ≥ 0.85 untuk pertanyaan sah.
5. Sistem berjalan 100% lokal tanpa biaya API, menggunakan Ollama dan llama3.2 (2 GB, muat di GPU GTX 1650).

5.2 Tantangan

1. Keterbatasan GPU: GTX 1650 4GB membatasi pilihan LLM ke model kecil ($\leq 3B$ parameter). Model lebih besar seperti llama3 (8B) jatuh ke CPU inference.
2. Python 3.14 compatibility: Python versi terbaru belum didukung penuh oleh PyTorch wheel stabil, memerlukan nightly build untuk CUDA.
3. Docling dependency: Library parsing PDF (Docling) besar (1 GB) dan hanya dibutuhkan saat ingest, tidak untuk runtime QA.
4. Evaluasi RAGAS terbatas: Evaluasi dengan RAGAS membutuhkan LLM judge, hasil bervariasi tergantung model yang digunakan. Retrieval accuracy tanpa LLM lebih objektif.
5. Bahasa Indonesia: BGE-M3 mendukung multilingual, namun akurasi retrieval untuk teks berbahasa Indonesia perlu pengujian lebih lanjut dengan dataset *benchmark*.